

**Institute of Computer Science and Digital Innovation**

**GROUP ASSIGNMENT**

[January 2025]

**CD 107- JAVA PROGRAMMING 2**

**Submitted by**

| Name | Student ID |
|---|---|
| Wai Yan Htet | 1002473399 |

## Introduction

Our School Management System is named UCSI+. UCSI+ is designed to be a comprehensive and user-friendly school management system, meeting the needs of modern educational institutions such as UCSI University.

## Objectives:

1. Create a centralized management system that combines student and teacher information into one system. This makes data management efficient and reduces extra administrative work.
2. Create user-friendly interaction with the users by providing a graphical user interface (GUI) using Java Swing to facilitate seamless data entry, record updates, and search functions.
3. Enhance data integrity by using encapsulation to protect sensitive data while allowing controlled access via getter and setter methods.

## Benefits:

1. UCSI+ improve efficiency by automating routine administrative tasks such as updating grades, assigning courses, and managing records, the system minimizes manual errors and saves time.
2. Enhanced Decision Making: Quick access to accurate student and teacher data aids school administrators in making informed decisions regarding academic planning and resource allocation.
3. UCSI+ is a cost-effective solution to automate administrative tasks and reduce the need for excessive administrative work and paper-based record keeping.

## Users:

1. School administrators: Responsible for overall school management, they use the system to maintain and update records, manage staff, and check student performance.
2. Teachers: They can update student grades, manage their subject, and check their salaries based on subjects taught.
3. Students: Students can check their grades and CGPA for their courses.

**Class Diagram with Explanations**

| **SchoolManagementGUI** |
| --- |
| main(args: String[]) |
| Main(args: String[]) : void |



| **MainFrame** |
| --- |
| students: ArrayList<Student><br>teachers: ArrayList<Teacher><br>addStudentButton: JButton<br>addTeacherButton: JButton<br>displayTeacherButton: JButton<br>displayTeacherButton: JButton<br>searchButton: JButton<br>exitButton: JButton |
| addStudent(): void<br>addTeacher(): void<br>displayTeacher(): void<br>displayStudent(): void<br>searchByID(): void |



| **Person** |
| --- |
| name: String<br>age: int<br>address: String<br>contact: String |
| getName(): String<br>setName(): void<br>getAge(): int<br>setAge(): void<br>getAddress(): String<br>setAddress(): void<br>getContact(): String<br>setContact(): void |



| **Student** | **Teacher** |
|---|---|
| studentID: String<br>course: String<br>grades: String<br>cgpa: double | teacherID: String<br>subject: String<br>salary: double<br>experience: String |
| getStudentID(): String<br>setStudentID(): void<br>getCourse(): String<br>setCourse(): void<br>getGrades(): String<br>setGrades(): void<br>getCgpa(): double<br>setCgpa(): void<br>getDetails(): String | getTeacherID(): String<br>setTeacherID(): void<br>getSubject(): String<br>setSubject: void<br>getSalary(): String<br>setSalary(): void<br>getExperience(): String<br>setExperience(): void<br>getDetails(): String |

**Explanations**

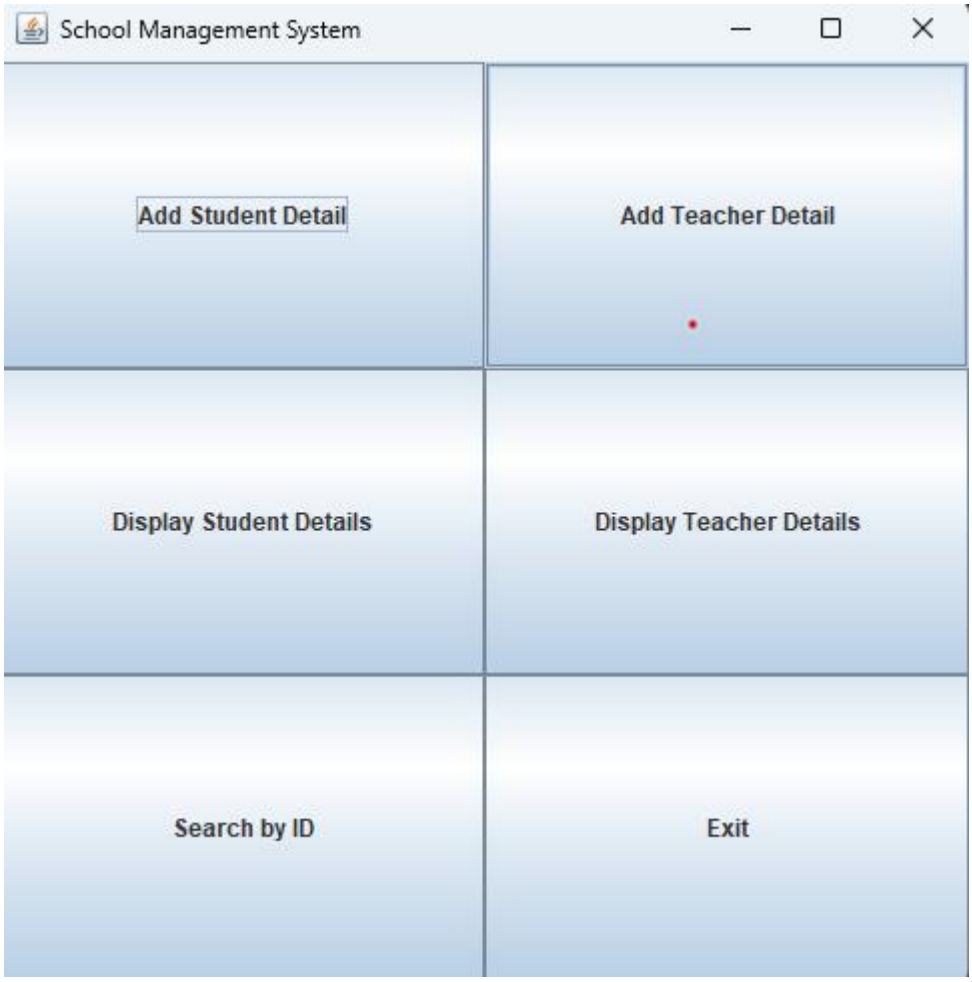
**Encapsulation** is a technique of hiding data and allowing controlled access through getter and setter methods. In these codes, the Person class has four private attributes, which are name, age, address, and contact number. For Student class, the private attributes include student ID, courses, grades, and CGPA. For Teacher class, the private attributes include teacher ID, subject, salary, and years of experience. Since attributes are private, public getter and setter methods can allow controlled access. The use of encapsulation hides the internal state of the classes from direct access, preventing unintended modifications and enhancing security.

**Inheritance** allows subclasses (Student and Teacher) to reuse the attributes and methods from the superclass (Person). Person class is a super class that contain the common attributes such as name, age, address, and contact number. It contains getter and setter methods for these attributes. On the other hand, Student class and Teacher class are the subclasses. Student extends Person and adds: student ID, courses, grades, and CGPA. Teacher extends Person and adds: teacherID, subject, salary, and experience. Both subclasses inherit the getName(), setName(), getAge(), etc. methods from Person. The use of Inheritance allows a child class to reuse methods and fields from a parent class, reducing code duplication and improving efficiency. It is easier to maintain as it makes updates easier and reducing errors.

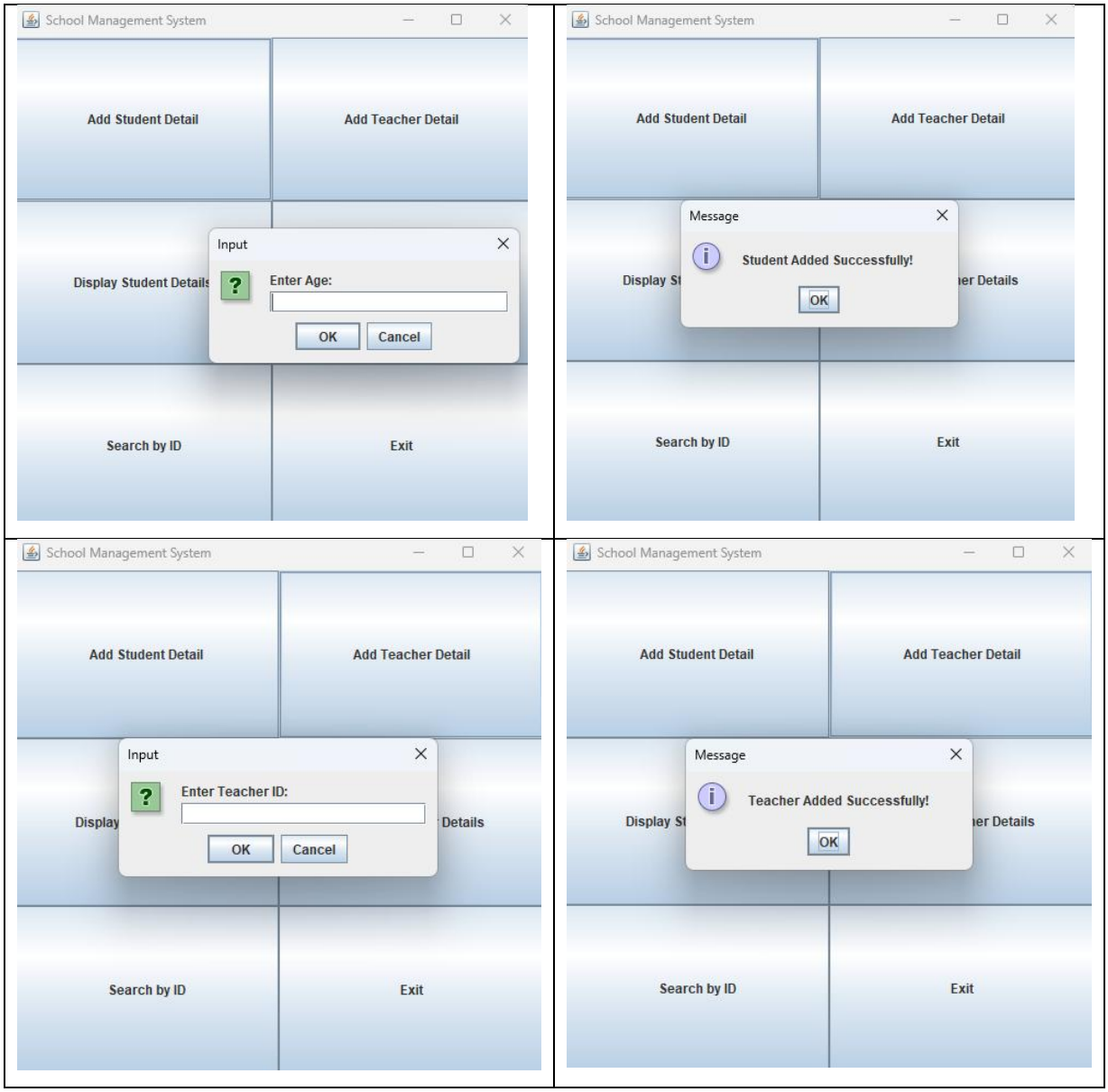
**Polymorphism** allows the same method name to behave differently depending on the context. For the overriding method, the getDetails() method is overridden in both Student and Teacher classes to return different information. This ensures that calling getDetails() on a Student object gives student details, while calling it on a Teacher object gives teacher details. For the overloading method, it states that multiple methods have the same name but different parameters in the same class. The example is searchByID method that contain two different parameters.

## Screenshot of the GUI

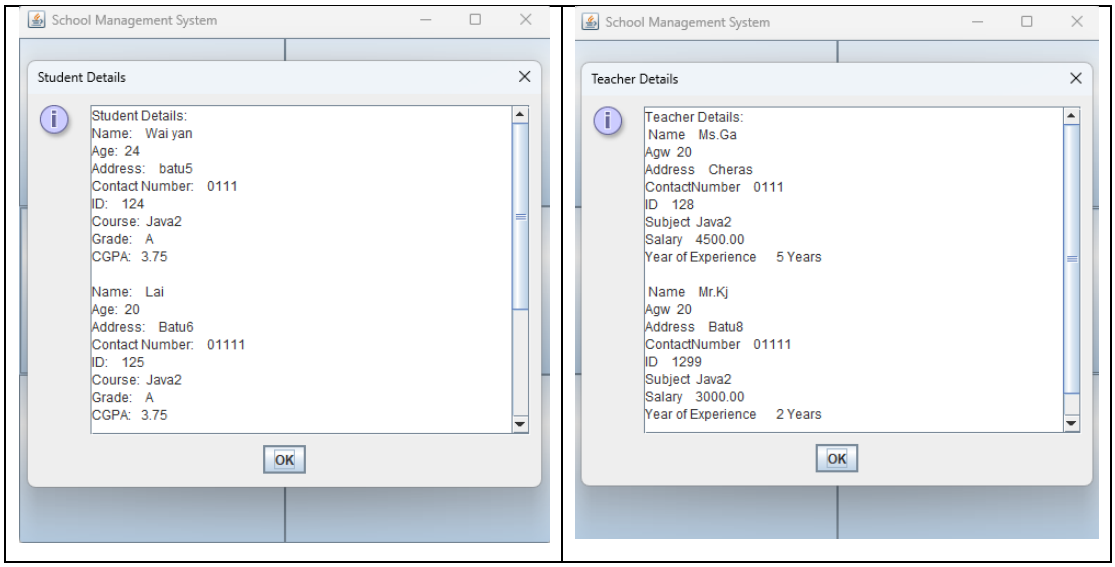
Main page and Functions of School Management System



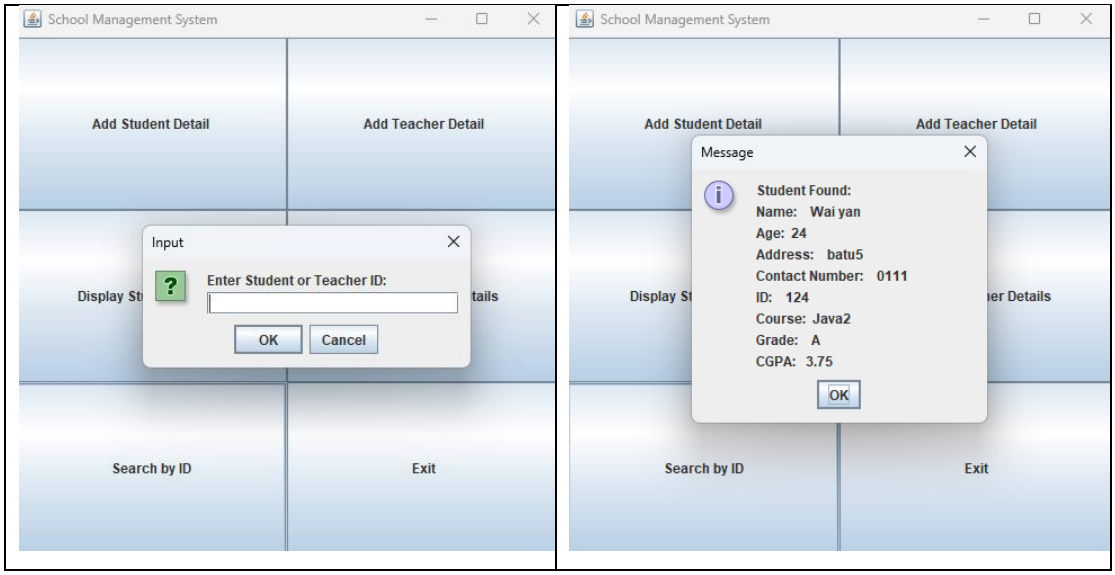
Input Data by using function



Display the input data




SearchByID function



## Code Implementation Details

The **School Management System** was implemented using **Java Swing for the GUI** and **Object-Oriented Programming (OOP) concepts** for modularity and scalability.

### 1. Graphical User Interface (GUI) Implementation

The GUI was designed using **JFrame** with a **GridLayout** to accommodate buttons for key functionalities.

**JButton components** handle actions such as adding, displaying, and searching records.

**JOptionPane dialogs** allow user input for adding new records.

**JTextArea inside JScrollPane** displays student and teacher details in a scrollable format.

### 2. Object-Oriented Approach

The system follows a structured approach with three main classes:

**Person (Super Class)**: Contains shared attributes like, age, address, and contact.

**Student (Subclass)**: Extends Person and adds specific attributes like studentID, course, grades, and cgpa.

**Teacher (Subclass)**: Extends Person and includes teacherID, subject, salary, and experience.

### 3. Data Handling and Validation

To ensure **data integrity and avoid duplication**, validation checks are performed before adding records:

**Unique ID Check**: Before adding a new student/teacher, the system verifies if the ID already exists.

**Input Validation**: Numeric fields such as age, cgpa, and salary are validated to prevent incorrect input.

### 4. Search and Display Functionality

**Search by ID**: Users can search for students or teachers using their ID.

**Displaying Data**: The system collects details from the ArrayList and presents them in a JTextArea within a JScrollPane for better readability.

**5. Exit Functionality**

A simple **System.exit(0);** is used to allow users to exit the application from the GUI.

**Challenges Faced & Solutions**

1. Student ID Duplication

Challenge:

When manually adding student details, users may accidentally insert two students with the same ID. This will cause data duplication when retrieving student details.

Solution:

We implement a validation check before adding a new student to ensure the Student ID is unique. If the user enters same Student ID again, it will show an error message.

2. Formatting CGPA to Two Decimal Places

Challenge:

Since CGPA is stored as a double, it can display too many decimal places (example: 3.8976543). This makes the output look messy.

Solution:

Use String.format() or DecimalFormat to round and limit the CGPA to two decimal places (example: 3.90).

3. Managing Input Data Efficiently

Challenge:

Handling multiple student and teacher records manually can be inefficient, especially if the data is stored temporarily and not managed in an organized manner. Without a proper data structure, searching, updating, and displaying records would be difficult.

Solution:

To efficiently store and manage the input data, an **ArrayList** is used. This allows dynamic storage of student and teacher records, making it easy to add, remove, and search for records based on Student ID or Teacher ID. The ArrayList also enables seamless iteration when displaying all stored data.

Person class

```
import java.sql.SQLOutput;


public class Person {

    private String name;

    private int age;

    private String address;

    private String contactNumber;


    // Constructor

    public Person(String name, int age, String address, String
contactNumber) {

        this.name = name;

        this.age = age;

        this.address = address;

        this.contactNumber = contactNumber;

    }



    public String getName() { return name; }

    public void setName(String name) { this.name = name; }


    public int getAge() { return age; }

    public void setAge(int age) {

        if (this.age > 0 && this.age < 120) {this.age = age; }

        else {

            System.out.println("Invalid age! Please enter a valid
age.");

        }

    }


    public String getAddress() { return address; }

    public void setAddress(String address) { this.address = address;
}


    public String getContactNumber() { return contactNumber; }
```

```
    public void setContactNumber(String contactNumber) {
this.contactNumber = contactNumber; }


    public String getDetails(){

        String D = "";


        D+= String.format("Name:     %s\n",getName());

        D+= String.format("Age:  %d\n",getAge());

        D+= String.format("Address:     %s\n",getAddress());

        D+= String.format("Contact Number:
%s\n",getContactNumber());




        return D;

    }

}
```

Student class

```
class Student extends Person {

    private String studentID;

    private String courses;

    private String grades;

    private double cgpa;


    // Constructor

    public Student(String name, int age, String address, String
contactNumber, String studentID, String courses, String grades,
double cgpa) {

        super(name, age, address, contactNumber);

        this.studentID = studentID;

        this.courses = courses;

        this.grades = grades;

        this.cgpa = cgpa;

    }
```

```
    // Overloaded constructor

    public Student(String studentID, String courses) {

        super("Unknown", 0, "Unknown", "Unknown");

        this.studentID = studentID;

        this.courses = courses;

        this.grades = grades;

        this.cgpa = 0.0;

    }

    public String getStudentID() { return studentID; }

    public void setStudentID(String studentID) { this.studentID =
studentID; }


    public String getCourses() { return courses; }

    public void setCourses(String courses) { this.courses = courses;
}


    public String getGrades() { return grades; }

    public void setGrades(String grades) { this.grades = grades; }


    public double getCgpa() { return cgpa; }

    public void setCgpa(double cgpa) { this.cgpa = cgpa; }

    // Overriding getDetails method

    @Override

    public String getDetails() {

        String S = "";


        S+= String.format("Name:     %s\n",getName());

        S+= String.format("Age:   %d\n",getAge());

        S+= String.format("Address:     %s\n",getAddress());

        S+= String.format("Contact Number:
%s\n",getContactNumber());

        S+= String.format("ID:     %s\n",getStudentID());

        S+= String.format("Course:   %s\n",getCourses());

        S+= String.format("Grade:     %s\n",getGrades());
```

```
        S+= String.format("CGPA:    %.2f\n",getCgpa());

        return S;}


}
```

Teacher class

```
class Teacher extends Person {
    private String teacherID;
    private String subject;
    private double salary;

    private String exp;
    // Constructor with all parameters
    public Teacher(String name, int age, String address, String
contactNumber, String teacherID, String subject, double salary,
String exp) {
        super(name, age, address, contactNumber);
        this.teacherID = teacherID;
        this.subject = subject;
        this.salary = salary;
        this.exp = exp;
    }

    // Overloaded constructor with only ID and subject
    public Teacher(String teacherID, String subject) {
        super("Unknown", 0, "Unknown", "Unknown");
        this.teacherID = teacherID;
        this.subject = subject;
        this.salary = 0.0;
        this.exp = "NA";
    }

    // Getters and Setters
    public String getTeacherID() { return teacherID; }
    public void setTeacherID(String teacherID) { this.teacherID =
teacherID; }

    public String getSubject() { return subject; }
    public void setSubject(String subject) { this.subject = subject;
}

    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }

    public String getExp(){ return exp;}
    public void setExp(String exp){this.exp = exp;}

    @Override
    public String getDetails() {
        String T =" ";
        T+= String.format("Name     %s\n",getName());
        T+= String.format("Agw   %d\n",getAge());
        T+= String.format("Address     %s\n",getAddress());
        T+= String.format("ContactNumber
%s\n",getContactNumber());
        T+= String.format("ID     %s\n",getTeacherID());
        T+= String.format("Subject   %s\n",getSubject());
```

```
        T+= String.format("Salary     %.2f\n",getSalary());
        T+= String.format("Year of Experience      %s\n",getExp());
        return T;
    }
}
```

MainProgram

```
import java.util.Scanner;


public class MainProgram {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);


        // Person Input

        System.out.println("Enter Person Details:");

        System.out.print("Name: ");

        String personName = scanner.nextLine();

        System.out.print("Age: ");

        int personAge = scanner.nextInt();

        scanner.nextLine();

        System.out.print("Address: ");

        String personAddress = scanner.nextLine();

        System.out.print("Contact Number: ");

        String personContact = scanner.nextLine();


        Person person = new Person(personName, personAge,
personAddress, personContact);

        System.out.println("\nPerson Details:\n" +
person.getDetails());


        // Student Input

        System.out.println("\nEnter Student Details:");

        System.out.print("Name: ");

        String studentName = scanner.nextLine();

        System.out.print("Age: ");

        int studentAge = scanner.nextInt();
```

```
        scanner.nextLine();

        System.out.print("Address: ");

        String studentAddress = scanner.nextLine();

        System.out.print("Contact Number: ");

        String studentContact = scanner.nextLine();

        System.out.print("Student ID: ");

        String studentID = scanner.nextLine();

        System.out.print("Courses: ");

        String studentCourses = scanner.nextLine();

        System.out.print("Grades (%): ");

        String studentGrades = scanner.nextLine();

        System.out.print("CGPA: ");

        double studentCgpa = scanner.nextDouble();

        scanner.nextLine(); // Consume newline


        Student student = new Student(studentName, studentAge,
studentAddress, studentContact, studentID, studentCourses,
studentGrades, studentCgpa);

        System.out.println("\nStudent Details:\n" +
student.getDetails());


        // Teacher Input

        System.out.println("\nEnter Teacher Details:");

        System.out.print("Name: ");

        String teacherName = scanner.nextLine();

        System.out.print("Age: ");

        int teacherAge = scanner.nextInt();

        scanner.nextLine();

        System.out.print("Address: ");

        String teacherAddress = scanner.nextLine();

        System.out.print("Contact Number: ");

        String teacherContact = scanner.nextLine();

        System.out.print("Teacher ID: ");
```

```
        String teacherID = scanner.nextLine();

        System.out.print("Subject: ");

        String teacherSubject = scanner.nextLine();

        System.out.print("Salary: ");

        double teacherSalary = scanner.nextDouble();

        System.out.println("Year of Experience");

        String teacherExp = scanner.nextLine();


        Teacher teacher = new Teacher(teacherName, teacherAge,
teacherAddress, teacherContact, teacherID, teacherSubject,
teacherSalary,teacherExp);

        System.out.println("\nTeacher Details:\n" +
teacher.getDetails());


        scanner.close();

    }

}
```

School Management System (GUI)

```
import javax.swing.*;

import java.awt.*;

import java.util.ArrayList;

import javax.swing.JFrame;


public class SchoolManagementGUI {

    public static void main(String[] args) {

        new MainFrame();

    }

}


class MainFrame extends JFrame {

    private ArrayList<Student> students = new ArrayList<>();

    private ArrayList<Teacher> teachers = new ArrayList<>();

    private ImageIcon uniIcon, uni1Icon;
```

```
    public MainFrame() {

        setTitle("School Management System");

        setSize(500, 500);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setLayout(new GridLayout(3,5));


        JButton addStudentButton = new JButton("Add Student Detail");

        JButton addTeacherButton = new JButton("Add Teacher Detail");

        JButton displayStudentsButton = new JButton("Display Student
Details");

        JButton displayTeachersButton = new JButton("Display Teacher
Details");

        JButton searchButton = new JButton("Search by ID");

        JButton exitButton = new JButton("Exit");


        add(addStudentButton);

        add(addTeacherButton);

        add(displayStudentsButton);

        add(displayTeachersButton);

        add(searchButton);

        add(exitButton);



        //Lamda expression

        addStudentButton.addActionListener(e -> addStudent());

        addTeacherButton.addActionListener(e -> addTeacher());

        displayStudentsButton.addActionListener(e ->
displayStudents());

        displayTeachersButton.addActionListener(e ->
displayTeachers());

        searchButton.addActionListener(e -> searchByID());

        exitButton.addActionListener(e -> System.exit(0));


        setVisible(true);

    }
```

```
    private void addStudent() {

        String studentID = JOptionPane.showInputDialog("Enter Student
ID:");



        for (Student s : students) {

            if (s.getStudentID().equals(studentID)) {

                JOptionPane.showMessageDialog(this, "Student ID
already exists!");

                return;

            }

        }


        String name = JOptionPane.showInputDialog("Enter Student
Name:");

        int age = Integer.parseInt(JOptionPane.showInputDialog("Enter
Age:"));

        String address = JOptionPane.showInputDialog("Enter
Address:");

        String contact = JOptionPane.showInputDialog("Enter Contact
Number:");

        String course = JOptionPane.showInputDialog("Enter Course:");

        String grades = JOptionPane.showInputDialog("Enter Grades:");

        double cgpa =
Double.parseDouble(JOptionPane.showInputDialog("Enter CGPA:"));


        students.add(new Student(name, age, address, contact,
studentID, course, grades, cgpa));

        JOptionPane.showMessageDialog(this, "Student Added
Successfully!");

    }



    private void addTeacher() {

        String teacherID = JOptionPane.showInputDialog("Enter Teacher
ID:");


        for (Teacher t : teachers) {

            if (t.getTeacherID().equals(teacherID)) {
```

```
                JOptionPane.showMessageDialog(this, "Teacher ID
already exists!");

                return;

            }

        }


        String name = JOptionPane.showInputDialog("Enter Teacher
Name:");

        int age = Integer.parseInt(JOptionPane.showInputDialog("Enter
Age:"));

        String address = JOptionPane.showInputDialog("Enter
Address:");

        String contact = JOptionPane.showInputDialog("Enter Contact
Number:");

        String subject = JOptionPane.showInputDialog("Enter
Subject:");

        double salary =
Double.parseDouble(JOptionPane.showInputDialog("Enter Salary:"));

        String TeacherExp = JOptionPane.showInputDialog("Enter Year
of Experience:");


        teachers.add(new Teacher(name, age, address, contact,
teacherID, subject, salary, TeacherExp));

        JOptionPane.showMessageDialog(this, "Teacher Added
Successfully!");

    }



    private void displayStudents() {

        if (students.isEmpty()) {

            JOptionPane.showMessageDialog(this, "No student details
available.");

        } else {

            StringBuilder details = new StringBuilder("Student
Details:\n");


            for (Student s : students) {

                details.append(s.getDetails()).append("\n");

            }
```

```
            JTextArea textArea = new JTextArea(details.toString());
            textArea.setEditable(false);
            JScrollPane scrollPane = new JScrollPane(textArea);
            scrollPane.setPreferredSize(new Dimension(400, 300));

            JOptionPane.showMessageDialog(this, scrollPane, "Student
Details", JOptionPane.INFORMATION_MESSAGE);
        }
    }

    private void displayTeachers() {
        if (teachers.isEmpty()) {
            JOptionPane.showMessageDialog(this, "No teacher details
available.");
        } else {
            StringBuilder details = new StringBuilder("Teacher
Details:\n");

            for (Teacher t : teachers) {
                details.append(t.getDetails()).append("\n");
            }

            JTextArea textArea = new JTextArea(details.toString());
            textArea.setEditable(false);
            JScrollPane scrollPane = new JScrollPane(textArea);
            scrollPane.setPreferredSize(new Dimension(400, 300));

            JOptionPane.showMessageDialog(this, scrollPane, "Teacher
Details", JOptionPane.INFORMATION_MESSAGE);
        }
    }

    private void searchByID() {
        String id = JOptionPane.showInputDialog("Enter Student or
Teacher ID:");
```

```
        if (id == null || id.trim().isEmpty()) {
            JOptionPane.showMessageDialog(this, "ID cannot be
empty!");
            return;
        }

        for (Student s : students) {
            if (s.getStudentID().equals(id)) {
                JOptionPane.showMessageDialog(this, "Student
Found:\n" + s.getDetails());
                return;
            }
        }

        for (Teacher t : teachers) {
            if (t.getTeacherID().equals(id)) {
                JOptionPane.showMessageDialog(this, "Teacher
Found:\n" + t.getDetails());
                return;
            }
        }

        JOptionPane.showMessageDialog(this, "No record found for ID:
" + id);
    }
}
```